**School of Engineering and Sciences**

# PROJECT REPORT

ON

---

# INVENTORY MANAGEMENT SYSTEM

---

*Under the Guidance of*

**DR. TIRHANKAR SENGUPTA**

**Department of Computer Science Engineering**

(Artificial Intelligence & Machine Learning)

**B.Tech. – CSE (AIML)**

**Batch: 2026 – 2029**


**TEAM MEMBERS**

| NAME | ROLL NUMBER |
|---|---|
| M. Tarakaram | AP25110070530 |
| H. Lokeswara Rao | AP25110070926 |
| P. V. Pavan Kumar | AP25110070596 |
| N. Manohar Chowdary | AP25110070805 |
| K. Arjun | AP25110070354 |


November 8, 2026

# ABSTRACT

Retail mobile phone and electronics showrooms manage products ranging from low-cost accessories to high-value smartphones. Maintaining inventory records manually or through disconnected spreadsheets frequently results in stock discrepancies, billing delays, unrecorded adjustments, and inaccurate profit calculations when unsold vendor purchases are mixed with monthly operating expenses. This report presents the design, implementation, and evaluation of MobileZone Pro, a web-based Inventory Management and Point-of-Sale (POS) System developed using Java 21, Spring Boot 3.2.5, Spring MVC, Spring Data JPA (Hibernate), Spring Security 6, Thymeleaf, and relational database storage (H2 and MySQL 8.0). The application unifies product catalog administration, multi-item supplier purchase orders, barcode-assisted POS billing, real-time stock movement tracking, accrual-based Cost of Goods Sold (COGS) and Profit-and-Loss (P&L) reporting, and a seven-role access control system within a single transactional architecture.

# CONTENTS

**Abstract** ii

**1 Introduction and Motivation** 1

1.1 Project Background and Industry Context . . . . . 1

1.2 Problems with Manual Inventory Systems and Stock Difficulties . . . . . 1

1.3 Sales and Purchase Tracking Challenges . . . . . 1

1.4 Motivation and Objectives of MobileZone Pro . . . . . 2

**2 Related Work** 3

2.1 Traditional Manual Systems and Existing Inventory Software . . . . . 3

2.2 Previous Research on Inventory Accuracy and Access Control . . . . . 3

2.3 Limitations of Existing Approaches and Project Contribution . . . . . 3

**3 System Architecture** 4

3.1 System Architecture Overview . . . . . 4

3.2 Main Modules and Component Communication . . . . . 4

3.3 Important Data Flows and Relational Schema . . . . . 4

**4 Implementation** 7

4.1 Technologies Used and Project Structure . . . . . 7

4.2 Product, Stock, and Purchase Management Implementation . . . . . 7

4.3 Sales, POS Billing, and Accrual P&L Business Logic . . . . . 8

**5 Experimental Results** 9

5.1 Functional Testing and Verification Test Cases . . . . . 9

5.2 Application Outputs and Financial Verification Results . . . . . 9

5.3 Discussion of Results . . . . . 9

**6 Conclusion and Scope of Future Work** 11

6.1 Project Summary and Main Objectives Achieved . . . . . 11

6.2 Benefits and Current Limitations . . . . . 11

6.3 Scope of Future Work . . . . . 11

**References (IEEE Format)** 12

**Appendix: Project Verification Artifacts & Links** **13**

# CHAPTER 1: INTRODUCTION AND MOTIVATION

## 1.1 Project Background and Industry Context

Inventory management governs how retail merchandise is procured from suppliers, stored on showroom shelves, monitored against minimum stock thresholds, and sold to customers. In a mobile phone and consumer electronics showroom, inventory control carries high operational and financial importance because the catalog spans both fast-moving accessories (chargers, cases, earphones, and power banks) and high-value smartphones priced above Rs. 1,00,000 per unit. Daily showroom operations require continuous coordination between catalog administration, supplier stock-in deliveries, counter billing, expense tracking, and periodic profitability review.

## 1.2 Problems with Manual Inventory Systems and Stock Difficulties

Many small and medium-sized retail showrooms rely on paper registers or standalone spreadsheets to record stock quantities and customer bills. As transaction volume grows, manual tracking introduces major issues:

- **Inventory Record Inaccuracy:** Updating sales and stock registers separately leads to transcription errors and omitted deductions, causing book stock to diverge from physical shelf stock.
- **Delayed Reorder Identification:** Without automated low-stock alerts, staff discover that popular items are depleted only when a walk-in customer requests the item at the counter.
- **Lack of Movement Audit Trail:** Overwriting a quantity cell in a spreadsheet erases the history of whether a stock change came from a purchase order, a customer sale, or a manual adjustment.
- **Loss of Historical Integrity:** Deleting discontinued products or overwriting unit purchase prices in flat files corrupts past sales records and distorts historical gross margins.

## 1.3 Sales and Purchase Tracking Challenges

Coordinating supplier purchases with counter sales presents both operational and accounting challenges. At the billing counter, cashiers must quickly look up items by barcode or SKU code, verify stock availability before confirming a bill, apply discounts, and generate a printable invoice while immediately deducting sold units. From an accounting perspective, basic store software often subtracts total monthly supplier purchases directly from monthly sales revenue. When a showroom procures a bulk shipment of smartphones for upcoming demand, treating unsold stock purchases as immediate expenses falsely reports a monthly loss. Accurate retail accounting requires capitalizing procured stock as an inventory asset and recognizing Cost of Goods Sold (COGS) strictly on the units sold during the reporting period.

## 1.4 Motivation and Objectives of MobileZone Pro

MobileZone Pro was developed to replace fragmented manual bookkeeping with a transaction-safe web application for mobile showrooms. The primary objectives of the system are: (1) maintain a centralized catalog with SKU codes, barcodes, soft deactivation, low-stock alerts, and a stock movement ledger; (2) execute supplier purchases (`PURCHASE_IN`) and POS sales (`SALE_OUT`) inside atomic database transactions that prevent overselling; (3) snapshot unit purchase cost at checkout (`SaleItem.costPrice`) to compute accurate accrual COGS, Gross Profit, and Net Profit; and (4) enforce multi-user security across seven predefined roles (Table 1).

**Table 1:** Predefined User Roles Implemented in MobileZone Pro (`UserRole.java`)

| **Role Identifier** | **Operational Scope in MobileZone Pro** |
|---|---|
| SUPER_ADMIN / ADMIN | Full system control, catalog, stock, purchases, sales, expenses, P&L reports, approvals, and users. |
| SHOWROOM_MANAGER | Executive dashboard, sales and stock monitoring, P&L analytics, activity logs, and approvals. |
| ACCOUNTANT | Operating expense management, sales/purchase ledger review, and P&L report export. |
| INVENTORY_MANAGER | Stock monitoring, low-stock alerts, manual stock adjustments, and movement ledger. |
| PURCHASE_OFFICER | Supplier management, multi-item purchase orders, and stock-in delivery recording. |
| SALESMAN (Cashier) | POS billing counter, barcode scanning, invoice printing, and own sales history. |

# CHAPTER 2: RELATED WORK

## 2.1 Traditional Manual Systems and Existing Inventory Software

Retail inventory control ranges from manual stock registers and spreadsheets to standalone billing programs and Enterprise Resource Planning (ERP) suites. Spreadsheets require no licensing cost but lack relational integrity, concurrent locking, and automatic stock deduction during billing, whereas enterprise ERP platforms introduce high cost and complexity for independent showrooms. In a multiple-case study of small retailers adopting inventory software, Ahn et al. [6] found that small stores achieve the highest operational efficiency when POS billing, customer history, and stock control are unified in one accessible interface.

## 2.2 Previous Research on Inventory Accuracy and Access Control

Operations management research has extensively studied Inventory Record Inaccuracy (IRI)—the mismatch between system stock records and physical shelf counts. Analyzing 369,560 inventory records across 37 retail stores, DeHoratius and Raman [1] found that 65% of system records differed from physical counts due to unlogged movements and checkout errors. DeHoratius, Mersereau, and Schrage [2] showed that automated reorder alerts fail unless physical recounts and transaction flows are reconciled in a stock audit ledger, while Heese [3] demonstrated that inaccurate cost and stock visibility distorts retail margins. Field studies by Hardgrave, Aloysius, and Goyal [4] and evaluations by Ishfaq and Raja [5] confirmed that combining barcode capture at receiving and checkout with cycle counts substantially reduces record discrepancy. For multi-user systems, Sandhu et al. [7] established the Role-Based Access Control (RBAC) model to prevent unauthorized changes.

## 2.3 Limitations of Existing Approaches and Project Contribution

Small-business inventory prototypes often exhibit three limitations: (1) overwriting item quantities in place without logging before-and-after stock movements; (2) subtracting bulk vendor purchases directly from monthly sales instead of computing accrual COGS on sold units; and (3) lacking role-based separation between cashiers, stock staff, and managers. Without claiming a novel algorithm, MobileZone Pro provides a practical software implementation that addresses these gaps via transactional stock movement logging [1], [2], dual-key barcode/SKU lookup [4], [5], historical cost snapshotting for accrual P&L reporting [3], and seven-role RBAC [7].

# CHAPTER 3: SYSTEM ARCHITECTURE

## 3.1 System Architecture Overview

MobileZone Pro follows a five-layer architecture implemented in Java 21 and Spring Boot 3.2.5 [8] under package `com.showroom.inventory`. As shown in Figure 1, HTTP requests pass through the Spring Security 6 filter chain to Spring MVC Controllers, which invoke transactional Service classes. The Service layer communicates with Spring Data JPA Repositories to persist and query entities in the relational database.

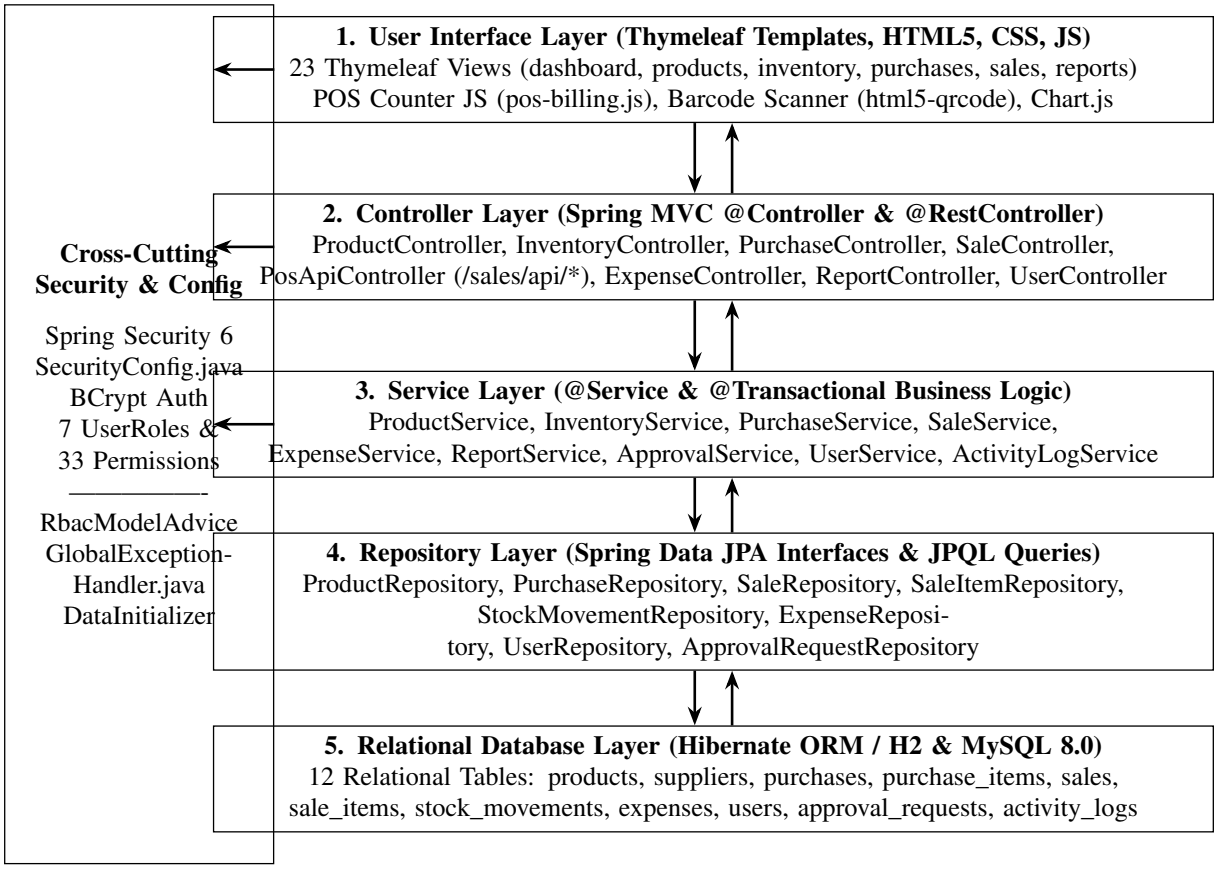



**Figure 1:** Five-Layer System Architecture of MobileZone Pro (Derived from Source Code)

## 3.2 Main Modules and Component Communication

Controllers exchange validated Data Transfer Objects (DTOs) with the presentation layer and domain entities with the service and repository layers. Table 2 summarizes the implemented layers and source components.

## 3.3 Important Data Flows and Relational Schema

Figure 2 depicts the three primary transactional workflows: (a) purchase order receiving in PurchaseService, which increments stock and logs `PURCHASE_IN`; (b) POS checkout in Sale-

**Table 2:** Architectural Layers and Verified Source Components

| Architectural Layer / Package | Implemented Components | Core Responsibility in Application |
|---|---|---|
| Presentation (templates/, static/) | 23 Thymeleaf HTML views, pos-billing.js, style.css | Renders UI views, collapsible sidebar, POS cart, and barcode scanner. |
| Security & Config (config/) | SecurityConfig, RbacModelAdvice, DataInitializer | Enforces BCrypt auth, 33 PERM_* URL guards, CSRF, and startup seeding. |
| Controller Layer (controller/) | 13 @Controller classes + PosApiController (@RestController) | Handles HTTP routing, DTO validation (@Valid), and JSON POS lookups. |
| Service Layer (service/) | 10 @Service classes (Product, Inventory, Purchase, Sale, Report) | Executes domain logic, stock checks, and COGS calculation (@Transactional). |
| Repository & Entity (repository/, entity/) | 12 JpaRepository interfaces & 12 @Entity classes | Maps domain entities to relational tables and runs JPQL queries. |

Service, which validates stock, snapshots unit costPrice, deducts stock, and logs `SALE_OUT`; and (c) P&L report generation in ReportService.

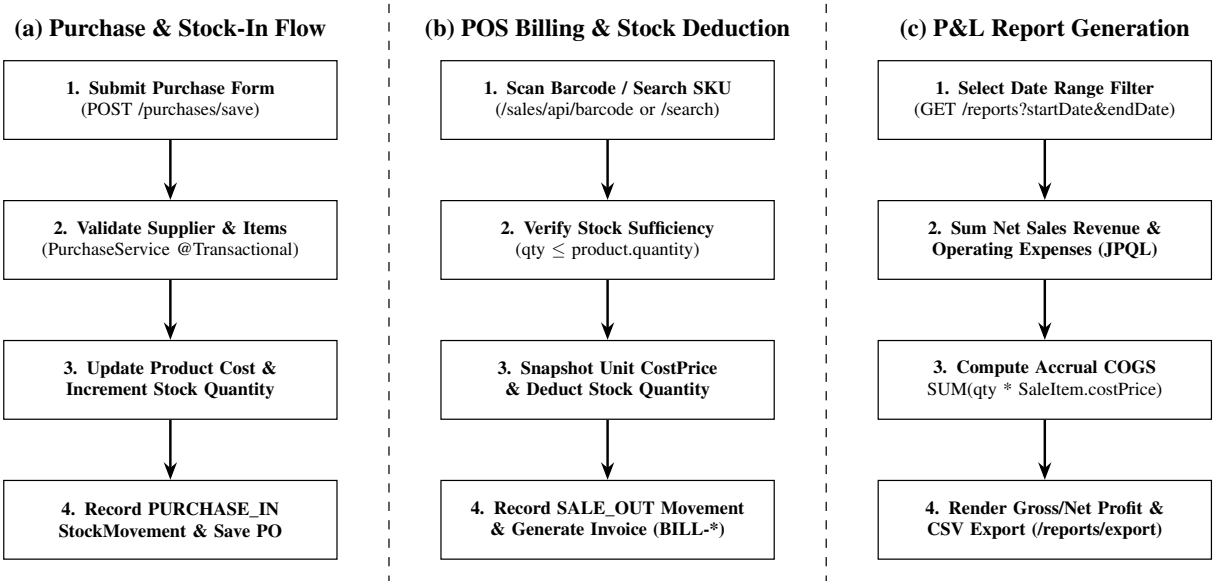



**Figure 2:** Operational Workflows for (a) Purchase Stock-In, (b) POS Billing, and (c) P&L Reporting

Figure 3 presents the simplified Entity-Relationship (ER) diagram of the relational database, combining the nine core operational tables in `db/schema.sql` with the three governance tables managed by Hibernate ORM.

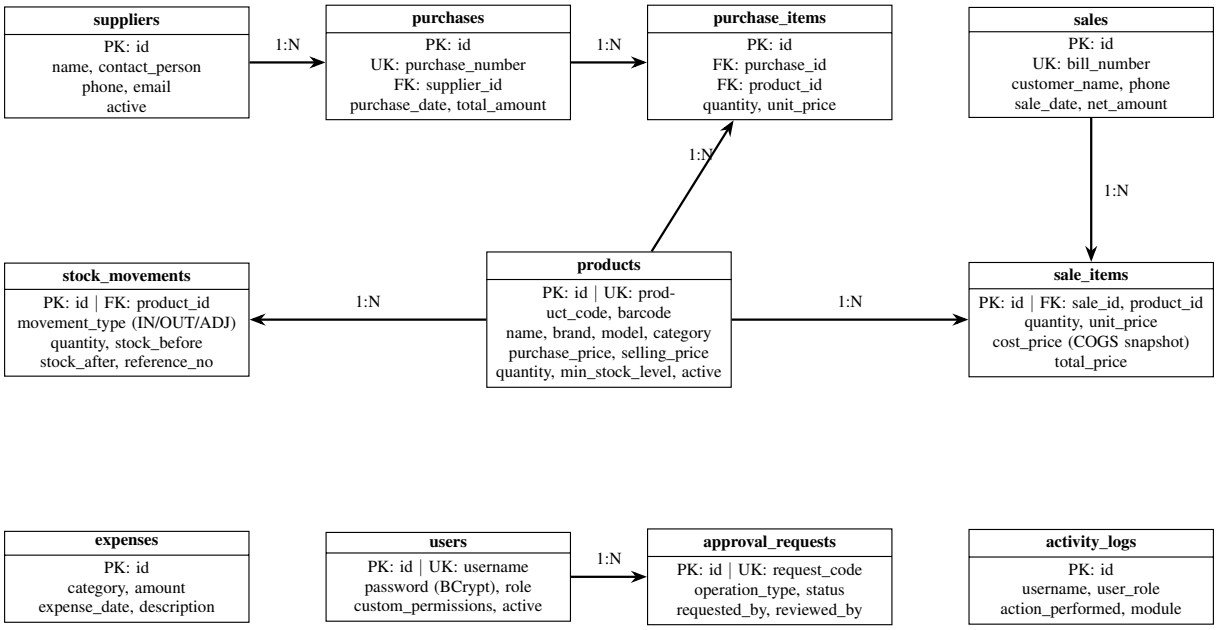



**Figure 3:** Simplified Entity-Relationship (ER) Diagram of the Relational Database Schema

# CHAPTER 4: IMPLEMENTATION

## 4.1 Technologies Used and Project Structure

The project is structured as a standard Maven application (`pom.xml`) targeting Java 21 and Spring Boot 3.2.5. Table 3 summarizes the verified technologies and libraries used across the backend and frontend.

**Table 3:** Verified Technology Stack (Inspected from `pom.xml` and `application.properties`)

| Component | Technology / Version | Implementation Role in MobileZone Pro |
|---|---|---|
| Backend Language & Framework | Java 21 LTS, Spring Boot 3.2.5 | Application runtime, dependency injection, embedded Tomcat server (port 8080). |
| Web & REST Layer | Spring Web MVC 6.1.6 | Server-side controllers and asynchronous JSON REST endpoints (/sales/api/*). |
| Persistence & ORM | Spring Data JPA, Hibernate 6.4.4 | Object-relational mapping, declarative @Transactional control, and JPQL queries. |
| Security Framework | Spring Security 6.2.4, BCrypt | Session authentication, password hashing, and 7-role / 33-permission access control. |
| Frontend & Barcode Scanning | Thymeleaf 3.1, Bootstrap 5.3, html5-qrcode | Server-rendered HTML views, POS cart script (pos-billing.js), camera/USB barcode scan. |
| Database Engine | H2 (File Mode) / MySQL 8.0+ | Persistent local database (./data/mobile_showroom_db) and MySQL DDL (db/schema.sql). |

## 4.2 Product, Stock, and Purchase Management Implementation

Product catalog management is implemented in `ProductController` and `ProductService`. Each `Product` entity maintains a unique SKU (`productCode`), EAN-13 barcode, category, `purchasePrice`, `sellingPrice`, `quantity`, `minStockLevel`, and an `active` flag. Deactivating a product sets `active = false` rather than executing a physical SQL DELETE, preserving foreign-key references in historical invoices. Barcode lookup in `ProductService.findActiveProductByBa` queries `Product.barcode` first and falls back to `Product.productCode`. Supplier restocking in `PurchaseService.createPurchase()` updates each item's `purchasePrice` and delegates stock updates to `InventoryService`, which logs before and after quantities in `StockMovement` (Listing 1).

**Listing 1:** Atomic Stock Deduction and Movement Logging (InventoryService.java, lines 74–90)

```
@Transactional
public void recordSaleStockDecrease(Product product, int quantity, String billNumber) {
    int before = product.getQuantity();
    if (before < quantity) {
        throw new InsufficientStockException(product.getName(), before, quantity);
    }
    int after = before - quantity;
    product.setQuantity(after);
    productRepository.save(product);
    StockMovement movement = StockMovement.builder().product(product)
        .movementType(StockMovementType.SALE_OUT).quantity(quantity)
        .stockBefore(before).stockAfter(after)
        .referenceNumber(billNumber).build();
    stockMovementRepository.save(movement);
```

```
}
```

## 4.3 Sales, POS Billing, and Accrual P&L Business Logic

POS billing is implemented across `SaleController`, `PosApiController`, `SaleService`, and `pos-billing.js`. When a cashier submits a bill to `POST /sales/save`, `SaleService.createSale()` validates stock sufficiency for all cart items before persisting changes and snapshots the product's current purchase cost onto `SaleItem.costPrice`. `ReportService.generateReport()` and `SaleItemRepository` then compute accrual Cost of Goods Sold (COGS), Gross Profit, and Net Profit over any selected date range (Listing 2).

**Listing 2:** POS Cost Snapshot and Accrual COGS Calculation (SaleService.java & SaleItemRepository.java)

```
// 1. SaleService.createSale(): Snapshot unit purchase price at the moment of sale
SaleItem item = SaleItem.builder().product(product)
    .quantity(itemDto.getQuantity()).unitPrice(unitPrice)
    .costPrice(product.getPurchasePrice())
    .totalPrice(unitPrice.multiply(BigDecimal.valueOf(itemDto.getQuantity())))
    .build();
inventoryService.recordSaleStockDecrease(product, itemDto.getQuantity(), billNo);

// 2. SaleItemRepository & ReportService: Compute Accrual COGS, Gross & Net Profit
@Query("SELECT COALESCE(SUM(si.quantity * COALESCE(si.costPrice, p.purchasePrice, 0)), 0) "
     + "FROM SaleItem si JOIN si.product p JOIN si.sale s "
     + "WHERE s.saleDate BETWEEN :start AND :end")
BigDecimal sumCostOfGoodsSoldBetween(LocalDateTime start, LocalDateTime end);

BigDecimal grossProfit = totalSalesRevenue.subtract(costOfGoodsSold);
BigDecimal netProfit = grossProfit.subtract(totalExpenses);
```

# CHAPTER 5: EXPERIMENTAL RESULTS

## 5.1 Functional Testing and Verification Test Cases

The application was compiled and tested against its persistent H2 database (`./data/mobile_showroom_db`) on Apache Tomcat (port 8080). Functional verification covered authentication, catalog management, purchase orders, barcode POS billing, stock adjustments, and financial reporting. Table 4 summarizes the primary test cases and actual application outputs.

**Table 4:** Functional Verification Test Cases Executed on MobileZone Pro

| Test ID | Module / Feature Tested | Test Input / Action Executed | Actual Observed Application Output | Status |
|---|---|---|---|---|
| TC-01 | RBAC Authentication | Login with 7 role accounts & invalid password | Valid logins routed to /dashboard; invalid rejected with error alert. | PASS |
| TC-02 | POS Barcode & SKU API | GET /sales/api/barcode?code=8901234560011 & MOB-IP15P | Returned HTTP 200 JSON for iPhone 15 Pro (price: Rs. 1,27,999, stock: 29). | PASS |
| TC-03 | Overselling Prevention | POST /sales/save requesting 999 units of MOB-S24U (stock: 10) | Aborted with InsufficientStockException; database stock remained 10. | PASS |
| TC-04 | POS Billing & Stock Out | Complete customer bill with discount at /sales/billing | Generated invoice BILL-*, deducted stock, logged SALE_OUT & costPrice. | PASS |
| TC-05 | Supplier Purchase Order | Submit multi-item restocking order at /purchases/new | Generated PO-*, incremented product quantities, logged PURCHASE_IN. | PASS |
| TC-06 | Low-Stock Alert Detection | Inspect /inventory?tab=lowstock for Anker Power Bank | Flagged ACC-ANK20K (stock: 1, min: 4) with global lowStockCount = 1. | PASS |
| TC-07 | Soft Product Deactivation | Deactivate catalog item via POST /products/deactivate/{id} | Marked item INACTIVE; hidden from POS while preserving past sales. | PASS |
| TC-08 | Accrual P&L & CSV Export | Generate report & CSV at /reports for Oct & Nov 2026 | Computed exact Revenue, COGS, Gross Profit, Expenses, and Net Profit. | PASS |

## 5.2 Application Outputs and Financial Verification Results

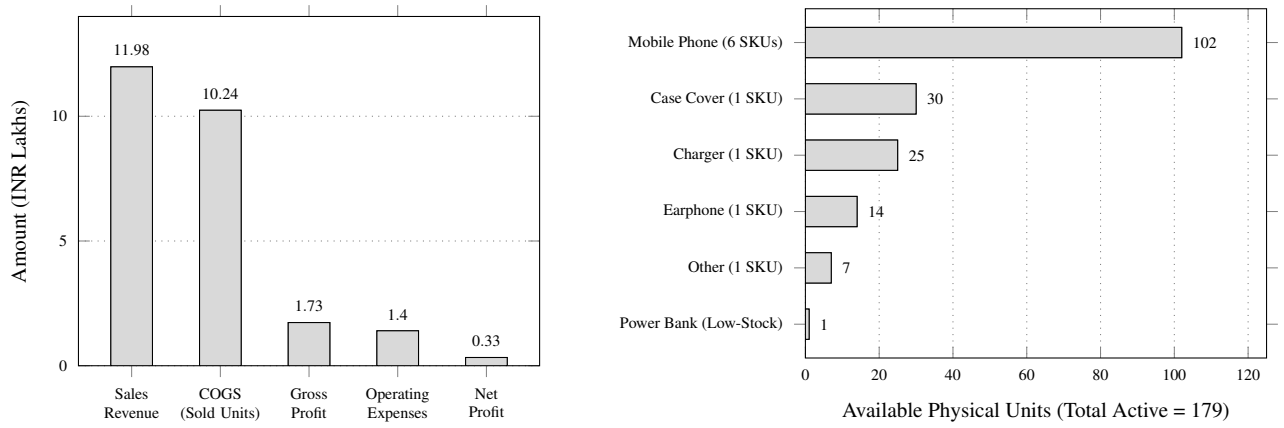
Table 5 presents the actual financial figures exported from `/reports/export` for the demonstration dataset across October 2026, November 2026, and the combined two-month period.

## 5.3 Discussion of Results

The experimental results confirm both transactional integrity and accounting accuracy. In November 2026 (Table 5 and Figure 4a), the showroom received Rs. 40,37,500.00 in sup-

**Table 5:** Observed Financial Statement Outputs Exported from Application Database

| Financial Metric | October 2026 | November 2026 | Combined (Oct–Nov) | Implemented Accounting Rule |
|---|---|---|---|---|
| Completed Sales Invoices | 1 Invoice (5 Units) | 11 Invoices (25 Units) | 12 Invoices (30 Units) | Count of Sales & sold units |
| 1. Sales Revenue (Net) | Rs. 3,14,797.00 | Rs. 11,97,502.00 | Rs. 15,12,299.00 | SUM(netAmount) after discounts |
| 2. Cost of Goods Sold (COGS) | Rs. 2,70,000.00 | Rs. 10,24,270.00 | Rs. 12,94,270.00 | SUM(qty * costPrice) of sold units |
| 3. Gross Profit (1 - 2) | Rs. 44,797.00 (14.23%) | Rs. 1,73,232.00 (14.47%) | Rs. 2,18,029.00 (14.42%) | Sales Revenue minus COGS |
| 4. Operating Expenses | Rs. 1,07,200.00 | Rs. 1,40,000.00 | Rs. 2,47,200.00 | SUM(amount) (Rent, Salary, etc.) |
| 5. Net Profit / Loss (3 - 4) | -Rs. 62,403.00 (Loss) | +Rs. 33,232.00 (Profit) | -Rs. 29,171.00 (Loss) | Gross Profit minus Expenses |
| 6. Inventory Purchases (POs) | Rs. 4,97,500.00 (1 PO) | Rs. 40,37,500.00 (5 POs) | Rs. 45,35,000.00 (6 POs) | Capitalized as stock asset |




**(a)** November 2026 Financial Summary

**(b)** Active Inventory Units by Category

**Figure 4:** (a) November 2026 Financial Summary and (b) Active Inventory Units by Category (Demonstration Dataset)

plier purchase orders (5 POs) while completing 11 customer invoices for 25 sold units totaling Rs. 11,97,502.00. Because `ReportService` calculates COGS strictly from those 25 sold units (Rs. 10,24,270.00), it reports an accurate Gross Profit of Rs. 1,73,232.00 (14.47%) and, after deducting Rs. 1,40,000.00 in monthly expenses, a Net Profit of Rs. 33,232.00 (+2.78%). Meanwhile, Figure 4(b) shows that the 179 active units across 11 active SKUs are tracked by category, with the single low-stock item (Anker Power Bank, 1 unit vs. minimum 4) flagged for reorder.

# CHAPTER 6: CONCLUSION AND SCOPE OF FUTURE WORK

## 6.1 Project Summary and Main Objectives Achieved

This project designed, implemented, and verified MobileZone Pro, a web-based Inventory Management and POS Billing System built with Java 21, Spring Boot 3.2.5, Spring Data JPA, Spring Security 6, and Thymeleaf. All primary project objectives were achieved: (1) manual registers were replaced by a 12-entity relational database with soft-delete catalog protection and an immutable stock movement ledger; (2) supplier purchase orders and barcode-enabled POS sales were enclosed within `@Transactional` service methods that prevent negative stock balances; (3) historical unit purchase cost was captured at checkout to deliver accurate accrual COGS, Gross Profit, and Net Profit statements; and (4) multi-user operations were secured using BCrypt authentication, seven predefined roles, 33 permissions, managerial approvals, and an activity log.

## 6.2 Benefits and Current Limitations

The system provides practical benefits for a retail showroom, including fast barcode/SKU billing, immediate visibility into low-stock and out-of-stock products, role-scoped navigation, and one-click CSV report exports. At the same time, three current limitations are noted: (1) the database models a single showroom without multi-branch stock transfers; (2) products are tracked by aggregate SKU quantity rather than individual 15-digit IMEI serial numbers; and (3) the inventory insights module uses deterministic threshold rules rather than statistical or machine-learning demand forecasting.

## 6.3 Scope of Future Work

1. **Device-Level IMEI Serialization:** Extend `PurchaseItem` and `SaleItem` with a dedicated serial/IMEI table to track individual smartphone serial numbers from supplier receipt through warranty claims.
2. **Predictive Demand Forecasting:** Integrate time-series forecasting models trained on historical sales velocity to recommend dynamic reorder thresholds before seasonal demand peaks.
3. **Multi-Branch Synchronization and GST e-Invoicing:** Support inter-branch stock transfers across multiple showroom locations and add HSN-wise CGST/SGST tax breakdowns with automated digital invoice delivery.

# REFERENCES

[1] N. DeHoratius and A. Raman, "Inventory record inaccuracy: An empirical analysis," *Management Science*, vol. 54, no. 4, pp. 627–641, Apr. 2008, doi: 10.1287/mnsc.1070.0789.

[2] N. DeHoratius, A. J. Mersereau, and L. Schrage, "Retail inventory management when records are inaccurate," *Manufacturing & Service Operations Management*, vol. 10, no. 2, pp. 257–277, Spring 2008, doi: 10.1287/msom.1070.0203.

[3] H. S. Heese, "Inventory record inaccuracy, double marginalization, and RFID adoption," *Production and Operations Management*, vol. 16, no. 5, pp. 542–553, Sept.–Oct. 2007, doi: 10.1111/j.1937-5956.2007.tb00279.x.

[4] B. C. Hardgrave, J. A. Aloysius, and S. Goyal, "RFID-enabled visibility and retail inventory record inaccuracy: Experiments in the field," *Production and Operations Management*, vol. 22, no. 4, pp. 843–856, July–Aug. 2013, doi: 10.1111/poms.12010.

[5] R. Ishfaq and U. Raja, "Empirical evaluation of IRI mitigation strategies in retail stores," *Journal of the Operational Research Society*, vol. 71, no. 12, pp. 1972–1985, 2020, doi: 10.1080/01605682.2019.1640592.

[6] E. Ahn, M. Andrews, N. Deckard, A. Iruku, J. Lee, J. Lue, and T. Schulz, "Implementation of customer relationship management and inventory control software: A multiple case study of small retailers," Gemstone Program Thesis, Univ. of Maryland, College Park, MD, USA, 2011. [Online]. Available: `http://hdl.handle.net/1903/11389`

[7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer (IEEE)*, vol. 29, no. 2, pp. 38–47, Feb. 1996, doi: 10.1109/2.485845.

[8] VMware, Inc., "Spring Boot 3.2, Spring Security 6.2, and Spring Data JPA Reference Documentation," 2024. [Online]. Available: `https://docs.spring.io/spring-boot/docs/3.2.5/reference/html/`

# APPENDIX: PROJECT VERIFICATION ARTIFACTS & LINKS

## Repository and Cloud Deployment Links

A matching public repository or deployment URL for MobileZone Pro was not located in the web search. The links below are official technology documentation, not project-hosted links.

## Project Verification Artifacts

| Artifact | Project Details | Evidence |
|---|---|---|
| Local database | ./data/mobile_showroom_db (H2) | Section 5.1 |
| Functional tests | Authentication, POS, stock, purchasing, low-stock alerts, and CSV export | Table 4 |
| Financial results | October–November 2026 demonstration data | Table 5 / Figure 4 |

## Official Technology Documentation

- Java SE 21: `https://docs.oracle.com/en/java/javase/21/`
- Spring Boot 3.2.5: `https://docs.spring.io/spring-boot/docs/3.2.5/reference/html/`
- H2 Database: `https://h2database.com/html/main.html`
- MySQL 8.0: `https://dev.mysql.com/doc/refman/8.0/en/`
- Apache Maven: `https://maven.apache.org/guides/`

## Execution and Demonstration

The report identifies Java 21, Spring Boot 3.2.5, Maven, Apache Tomcat on port 8080, and persistent H2 storage. These are standard Maven commands; project-specific profiles may vary.

```
$ mvn clean package
$ mvn spring-boot:run
```

## Demo Account Credentials

Demo usernames and passwords were not included in the supplied report, so none are reproduced here.